

Weather Predictor — Tutorial de progreso

Autor: José Rubio · **Fecha:** 2026-04-22 · **Versión:** checkpoint pre-demo (rev 3)

1. Resumen ejecutivo

Weather Predictor es una aplicación educativa que predice la temperatura máxima diaria en cinco ciudades curadas — **KPHX (Phoenix)**, **KLAX (Los Angeles)**, **KLAS (Las Vegas)**, **KLGA (New York)** y **KBOS (Boston)** — y la compara contra los precios del mercado de contratos de eventos de **Robinhood** para esa misma temperatura. El objetivo no es apostar con dinero real, sino medir cuánto "edge" tiene un modelo ensemble propio frente al mercado y cómo mejora esa precisión con el paso de las horas del día. El scope se curó en torno a lo que Robinhood cubre con mercado líquido: 3 ciudades del Southwest (clima estable, alto poder predictivo) + 2 de la Costa Este (régimen más variable, para balancear la cartera).

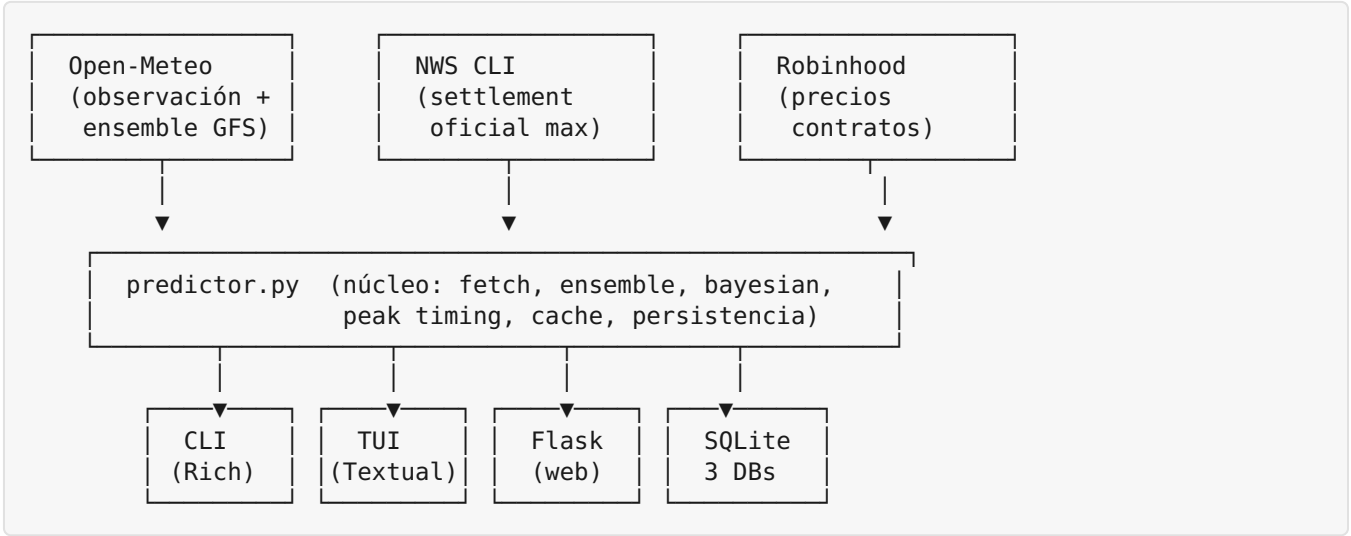
Hay tres interfaces (CLI, TUI terminal y web Flask), un polling automático cada 10 minutos y toda la información se persiste en SQLite para poder hacer análisis retrospectivo (Brier scores, reliability, histórico).

2. Objetivo del proyecto

- **Predecir** el máximo diario en °F con una distribución de probabilidad, no un solo número.
- **Comparar** esa distribución contra los precios de Robinhood en las 5 ciudades curadas.
- **Medir** si el modelo es consistente, calibrado, y si ofrece ventaja sistemática frente al mercado.
- **Identificar** días "difíciles" (fronts, anomalías) donde lo seguro es saltar la apuesta.
- **Aprender** sobre ensemble forecasting, calibración probabilística y prediction markets sin usar dinero real.

El diseño apunta a algo usable en el día a día: abro el iPad vía Tailscale por la mañana, miro `/cross` y decido qué ciudad tiene la mejor combinación de edge + dificultad baja para el día.

3. Arquitectura



- **Polling adaptativo:** un thread de background refresca la observación + ensemble cada 10 minutos **fuera del pico** y cada **3 minutos durante la ventana de pico** (12–17h local según estación). Durante pico también se invalida el cache de observaciones para leer el METAR fresco.
- **Cache:** TTL de 10 min sobre llamadas a Open-Meteo para evitar rate-limits (se bypassa durante pico).
- **Persistencia:** tres bases SQLite — `market_cache.db` (precios mercado + nuestro p por bin), `calibration.db` (snapshots de predicciones, outcomes diarios, resúmenes, bets simulados), y el archivo de climatología (30 años).
- **Acceso remoto:** la web corre en puerto 8000 y es visible desde el iPad por Tailscale (100.x.x.x) o la LAN local.

4. Fuentes de datos

Fuente	Uso	Formato
Open-Meteo Current	Temperatura actual, dewpoint, humedad, viento, presión	API JSON, llamada cada 10 min
Open-Meteo Ensemble GFS	31 miembros del modelo GFS, cada hora hoy/ mañana/pasado mañana	API JSON, temperatura + precipitación + nieve
Open-Meteo Archive	30 años de max diarios para la estación	API JSON, cacheado en SQLite localmente
NWS Climate Report (CLI)	Max oficial del día (settlement ground truth)	Scraping texto plano NWS
Robinhood event pages	Precios yes/no por threshold de temperatura	Scraping del <code>__NEXT_DATA__</code> JSON embebido

Ninguna requiere autenticación. Robinhood no expone API pública pero la página pública del contrato incluye los precios en un blob JSON que parseamos.

5. Modelo predictivo

La idea central: en vez de predecir un solo número, generar **una distribución de probabilidad** sobre cuál va a ser el máximo del día.

5.1 Ensemble GFS (baseline)

Open-Meteo ofrece los 31 miembros del ensemble GFS por separado. Cada miembro es una simulación del modelo con pequeñas perturbaciones en las condiciones iniciales. Para cada miembro extraemos el max del día, y eso nos da 31 muestras de la distribución predictiva. Calculamos p10, mediana y p90 directamente.

5.2 Reweight bayesiano (Fase 3c) + σ temporal

A medida que avanza el día, ya sabemos cuál fue la temperatura real las primeras horas. Esa observación nos permite reweightear los miembros del ensemble: los miembros que "se parecen" a lo observado reciben más peso; los que divergieron mucho, menos.

Implementación: softmax sobre la suma de errores cuadrados (SSE) estandarizada por un **σ dependiente de la hora** — horas dentro del pico usan $\sigma=1.5^\circ\text{F}$ (más exigente, más discriminación), horas adyacentes $\sigma=2.0$, medias $\sigma=2.5$, lejanas $\sigma=3.5$. Esto hace que un error de 2°F a las 14:00 pese mucho más que el mismo error a las 7:00. Re-muestreamos a $N=500$. El reporte muestra "eff N" (cuántos miembros efectivos quedan tras el reweight).

5.2.1 Detector de ruptura de régimen

Si la observación de una hora cae fuera de la envolvente p1–p99 del ensemble (con margen de 4°F para absorber sesgos sistemáticos de estaciones urbanas desérticas), esa hora se marca como **ruptura**. Con ≥ 2 horas rotas:

- se fuerza la dificultad del día a 100 ("ruptura de régimen")
- se envía push notification vía ntfy
- se recomienda saltar el día en `/cross`

El detector expone los frentes fríos/cálidos o errores de inicialización del modelo que harían el reweight ineficaz.

5.3 Peak timing (Fase 4)

Además del valor del máximo, predecimos **cuándo** va a ocurrir. Para cada miembro extraemos la hora del max y reportamos la hora modal, percentiles p10/p50/p90, P(ya ocurrió) y P(ocurrirá en las próximas 1/2/3/6 horas).

5.4 Calibración isotónica (pendiente activar)

Con los snapshots históricos (predicted_p, outcome) corremos **Pool-Adjacent-Violators** para aprender un mapa monótono $p_{\text{raw}} \rightarrow p_{\text{cal}}$. El gate actual exige ≥ 20 pares y ≥ 7 días distintos; con los datos actuales (3 días, 286 samples) el fit colapsa a un solo bloque $\rightarrow$ marcamos como "poco confiable" y no se aplica en vivo. Queda listo para activarse automáticamente cuando acumule más días.

6. Integración con el mercado (Robinhood)

Robinhood publica contratos "greater than N°F" por cada grado entero alrededor del pico esperado. Los parseamos de la página pública y convertimos a bins per-grado (compatibles con el formato Kalshi original).

6.1 /comparison — Robinhood vs modelo

Tabla bin por bin con los precios yes_mid de Robinhood y nuestra probabilidad derivada del ensemble. Barra visual y diff en pp.

6.2 /ladder — Decision ladder

Para cada threshold de temperatura ($\pm N$ alrededor de la mediana):

- yes/no nuestro vs yes/no Robinhood
- edge en puntos porcentuales
- EV al apostar \$1 a cada lado: $EV_{yes} = (p-k)/k$, $EV_{no} = (k-p)/(1-k)$
- Kelly fraction por lado: $f_{yes} = (p-k)/(1-k)$, $f_{no} = (k-p)/k$
- Pill **YES/NO** señalando el lado recomendado

Sirve para ver a simple vista qué thresholds tienen edge accionable.

6.3 /edge y /movement

- **Edge tracking**: lista los contratos con mayor edge actual y un análisis histórico (edges por bucket, ROI hipotético).
- **Movement**: evolución temporal del yes_mid Robinhood vs nuestro our_p para un bin dado — permite ver si el mercado converge a nosotros, si nosotros convergemos al mercado, o si divergen.

6.4 /bets — Simulador P&L

Cuando $|edge| \geq 5pp$, el sistema "apuesta" automáticamente \$10 hipotéticos en el lado correcto (yes si $our_p > kalshi$, no al revés), con guardias contra precios ≤ 0.01 o ≥ 0.99 . Al settlearse el día, calcula payoff real. Vista con n, win rate, stake total, P&L, ROI.

Dedupe por (station, date, ticker): solo registra el primer edge visto por ticker por día.

6.5 /calibration + /history

- **Calibration**: reliability diagram dual (nuestro vs Robinhood) + Brier score per-bin para que sean comparables.
- **History**: tabla diaria con max observado, Brier nuestro, Brier Robinhood, mejor edge y si fue correcto. Agregado: cuántos días ganamos al mercado.

6.6 Push notifications (ntfy)

Opt-in vía variable de entorno `NTFY_TOPIC`. Al detectar $|edge| \geq 10pp$ manda push al iPad; al settlearse un día, manda resultado con Briers comparados.

7. Guía por página

La web corre en `http://100.122.62.70:8000` (Tailscale) o `http://<LAN-ip>:8000`. Todas las páginas se actualizan cuando el poll loop termina un ciclo (cada 10 min).

Para decidir si apostar hoy

- **/ (dashboard principal)** — Todo lo importante del día en una pantalla: temperatura actual, pronóstico, distribución del ensemble, **badge de dificultad del día**, **dropdown para cambiar entre KPHX/KLAX/KLAS sin reiniciar**, cards de clima/viento/presión, peak timing, precipitación, y slots de aserciones personalizadas.
- **/cross** — Las 3 ciudades lado a lado con **ranking 1-2-3** y una **pill de recomendación** arriba: o bien "apuesta a STATION SIDE +Xpp" (cuando el #1 tiene edge $\geq 5pp$ y dificultad < 30) o "△ mejor saltar hoy". Selector hoy/mañana/pasado.
- **/ladder** — Para cada temperatura umbral (ej. $>80^\circ$, $>82^\circ$...): nuestro yes/no vs el de Robinhood, edge en pp, EV% al apostar \$1, Kelly fraction, y una pill YES/NO con el lado recomendado. Selector de ventana $\pm 2/\pm 3/\pm 4/\pm 6/\pm 10$ /todo.
- **/edge** — Tabla con los edges disponibles ahora ordenados por magnitud + performance histórica por bucket de edge (si edges $> 5pp$ ganaron dinero hipotético o no).
- **/comparison** — Barra visual bin-por-bin: yes_mid de Robinhood vs nuestra probabilidad + diff en pp.

Para entender el clima

- **/timing** — A qué hora se espera el pico de temperatura: hora modal, rango p10-p90, P(ya ocurrió), P(en las próximas 1/2/3/6h).
- **/precip** — Probabilidad de lluvia/nieve para hoy/mañana/pasado: P(any), P(notable), P(heavy), mm esperados, P(nieve).

Para ver si el sistema funciona

- **/reweight** — Diagnóstico hora por hora del reweight bayesiano: obs real vs ensemble p10/p50/p90, σ aplicado en esa hora (verde cuando está en la ventana de pico), n miembros matcheados, y flag de si la obs cayó dentro/fuera de p1–p99. Las filas de ruptura se marcan en rojo. Sirve para detectar sesgos sistemáticos del modelo y validar que el detector de régimen funciona.
- **/calibration** — Reliability diagram dual: cuando decimos 70%, ¿realmente pasa 70% de las veces? Muestra nosotros (azul) + Robinhood (rosa) + curva calibrada isotónica (verde, cuando aplique). Brier score crudo vs calibrado.
- **/history** — Tabla diaria con max observado, Brier nuestro vs Robinhood, mejor edge, si fue correcto. Total de días que ganamos al mercado.
- **/bets** — Simulador P&L: "si hubiera apostado \$10 cada vez que aparece edge $\geq 5pp$, ¿cuánto habría ganado?". Muestra n bets, win rate, stake total, payoff, ROI.
- **/movement** — Cómo evoluciona el precio de Robinhood vs nuestro pronóstico a lo largo del día para un bin específico. Útil para ver si el mercado converge a nosotros o divergimos.

Utilidades

- **/notify** — Configuración de push notifications (ntfy.sh). Envía alerta al iPad/móvil cuando aparece un edge grande ($\geq 10pp$) o cuando se settlea el día.
 - **/export** — Descarga CSV de cualquiera de las 5 tablas (snapshots, market_prices, day_summary, day_outcomes, simulated_bets) filtrado por estación y/o fecha. Para análisis en Excel.
 - **/status** — Salud del sistema: último poll OK, errores recientes, edad de los datos.
 - **/about** — Este mismo tutorial renderizado en HTML + link al PDF descargable.
-

8. Decisiones abiertas (invito feedback)

1. **Robinhood scraping vs IBKR ForecastEx API.** Hoy scrapeo el HTML público de Robinhood. Funciona, pero es frágil si cambian la estructura. IBKR ForecastEx (el backend probable) ofrece API pero pide cuenta IBKR + OAuth. ¿Vale la pena migrar?
 2. **Cross-station Bayesian reweight.** Extensión natural de Fase 3c: usar observaciones matutinas de KEWR, KJFK, KJRB (estaciones cercanas a KLGA) para reweightear el ensemble de KLGA. No implementado. ¿Prioridad?
 3. **Manual bet entry.** Hoy solo hay auto-bet cuando se detecta edge $\geq 5pp$. ¿Tiene valor permitir registrar bets manuales para llevar un portfolio personal?
 4. **Más features del ensemble.** Ya se expone lluvia/nieve. Quedan: viento max, nubosidad, probabilidad de ráfagas fuertes. ¿Interés?
 5. **Calibración en vivo.** Está implementada la isotónica pero no se aplica todavía en producción (gate de ≥ 7 días). Cuando se active, se aplicará a ladder, comparison y a la evaluación de aserciones. ¿Algún cambio en el gate?
 6. **Otras estaciones/mercados.** Hoy scope = KPHX/KLAX/KLAS/KLGA/KBOS (Robinhood). Robinhood cubre ~23 ciudades; los cálculos funcionan para cualquiera que Robinhood liste. ¿Expandir el scope curado o mantener estas 5?
-

9. Próximos pasos propuestos (ordenados por ROI)

Completados desde la revisión previa (rev 1 → rev 2):

- ☒ Parametrización Robinhood/Kalshi → `_market_name()` dinámico
- ☒ Ruta `/about` con tutorial accesible + `/tutorial.pdf`
- ☒ Pre-warm del cache de `/cross` (20s → 1.6s)
- ☒ Tests unitarios (52 tests: isotonic, kalshi math, robinhood, difficulty, predictor)
- ☒ Scope curado de 5 ciudades (KPHX/KLAX/KLAS + KLGA/KBOS para Costa Este)
- ☒ Polling adaptativo (3 min en pico, 10 min fuera) + invalidación de cache en pico
- ☒ σ temporal en reweight bayesiano (1.5°F en pico → 3.5°F lejos)
- ☒ Detector de ruptura de régimen (obs fuera de p1-p99 con margen 4°F) + push alert
- ☒ Panel `/reweight` con diagnóstico hora por hora
- ☒ Dropdown para cambiar estación sin reiniciar

-  Score de **difficultad del día** (spread + eff_N + clim + precip)
-  /cross con ranking 1-2-3 + pill de recomendación "apuesta a X / saltar hoy"

Pendientes:

1. **Validación de umbrales de dificultad con datos reales.** Los umbrales actuales (≥ 75 muy difícil, ≥ 55 difícil, ≥ 30 normal) son heurísticos. Después de acumular ~2 semanas de datos, medir si días "fácil" realmente tienen mejor Brier que días "difícil" y recalibrar.
 2. **Activación gradual de calibración isotónica** conforme acumulemos días settleados (auto-activación cuando pase el gate de 7 días).
 3. **Dificultad completa en /cross** (actualmente light: sólo spread + eff_N; falta añadir clim + precip que ya se usan en el badge del index).
 4. **Cross-station reweight** (usar obs matutinas de estaciones cercanas para reweightear el ensemble de la estación objetivo).
 5. **Histórico de dificultad vs accuracy** — tabla/gráfica que correlacione score del día con Brier real.
-

Apéndice: métricas mencionadas

- **Brier score:** error cuadrático medio entre probabilidad predicha y outcome binario (0 o 1). Menor = mejor. Comparable entre predictores si usan el mismo universo de preguntas.
- **Reliability diagram:** para cada bucket de probabilidad predicha, qué fracción de esos eventos ocurrieron. Si el punto cae en la diagonal, está calibrado.
- **Kelly fraction:** fracción óptima del bankroll a apostar para maximizar crecimiento logarítmico esperado. Requiere saber tu probabilidad verdadera (nunca la sabes exactamente).
- **Edge en pp (puntos porcentuales):** diferencia entre tu probabilidad y la del mercado. Positivo = mercado subestima.
- **eff N:** número efectivo de miembros del ensemble tras un reweight bayesiano. Si baja mucho respecto a 31, el ensemble no capturó bien la realidad reciente.